Quick Start

The example code provided in this quick start guide is for educational and demonstration purposes only. It may not represent best practices for production use.

Breaking Changes Notice

If you've just updated the package, it is recommended to check the *changelogs* for information on breaking changes.

Prerequisites

- Unity Version: Unity 6.0+
- Importer Dependencies (Install at least one):
  - glTFast for GLTF/GLB support.
  - UniGLTF for alternative GLTF/GLB support.
  - UniVRM for VRM support.
- Other Optional Dependencies:
  - Unity's Animation module for animator post-processing.

Key Concepts

- **Data Loaders**: Fetch raw avatar data (model, metadata, renders) from files or URIs.
- **Importers**: Convert raw data into Unity GameObjects.
- **Post-Processors**: Customize loaded avatars (e.g., add animators, override materials).
- **AvFlow**: Chains loaders, importers, and post-processors for a complete loading pipeline.

Avatars require a specific structure:

- Model file (e.g., `model.glb`, `model.vrm`).
- `metadata.json` with ID, type, gender, etc.
- Optional renders: `full.jpg/png`, `bust.jpg/png`.

Basic Usage

Use `AvFlow` to build a loading pipeline.

Example: Load from Local File

```
public class AvatarLoader : MonoBehaviour
{
    async void Start()
    {
```

```
        // FileAvDataLoader supports auto-detection of the model format by
file extension
        var flow = new AvFlow.Builder()
            .From(new FileAvDataLoader("path/to/avatar/dir"))  // Directory with
model, metadata, renders
            .Using(new GLTFastAvImporter())  // Or UniGLTFAvImporter,
UniVRM10AvImporter
            .Build();

        LoadedAv? avatar = await flow.TryRunAsync();  // Or `RunAsync()` for
explicit throwing errors.
        if (avatar != null)
        {
            avatar.GameObject.transform.position = Vector3.zero;  // Position
in scene
            // Use avatar.GameObject, avatar.Metadata, etc.
        }
        else
        {
            Debug.LogError("Failed to load avatar");
        }
    }
}
```

Example: Load from URI (Remote)

```
var flow = new AvFlow.Builder()
    .From(new URIAvDataLoader(
        "https://example.com/model.glb",
        "https://example.com/metadata.json",
        AvModelFileExtension.GLB // Model format must be explicitly declared
    ))
    .Using(new GLTFastAvImporter())
    .Build();

LoadedAv avatar = await flow.RunAsync();  // Throws on failure
```

Adding Post-Processors

Post-processors run after import. Synchronous ones run first, then async.

```
// In AvFlow.Builder
.ProcessWith(new AnimatorPostProcessor() { AnimatorController = myController })  //
Add animator
```

```
.ProcessWithAsync(new CachePostProcessor("cache/dir"))  // Cache downloaded model to local dir
```

- **Material Overrides/Shaders**: Use `MaterialOverridePostProcessor` or `ShaderSwapPostProcessor` with configs (create via ScriptableObject).
- **Conditional**: Wrap with `ConditionalPostProcessor` for logic-based processing.

Sync processors (e.g., AnimatorPostProcessor) always run before async ones (e.g., CachePostProcessor), regardless of addition order.

The `AvPost` class has static methods for fluent-style creation of post-processors.

Multi-Importer Example with More Post-Processors

```
[SerializeField] RuntimeAnimatorController _animator;
[SerializeField] Avatar _femAvatar;
[SerializeField] Avatar _mascAvatar;
[SerializeField] bool shouldLoadGltf;

new AvFlow.Builder()
.From(
    new FileAvDataLoader(
        shouldLoadGltf
        ? Path.Join(Application.streamingAssetsPath, "GLTFModelFolder")
        : Path.Join(Application.streamingAssetsPath, "VRMModelFolder")
    )
).Using(
    new UniVRM10AvImporter(),
    new GLTFastAvImporter(),
    new UniGLTFAvImporter()
    {
        ImporterContextSettings = new UniGLTF.ImporterContextSettings(invertAxis: UniGLTF.Axes.X) // For compatability with glTFast
    }
).ProcessWith(
    AvPost.ConfigureAnimator(_animator, _mascAvatar, new Dictionary<AvGender, Avatar>()
    {
        { AvGender.Feminine, _femAvatar },
        { AvGender.Masculine, _mascAvatar },
    }),

    AvPost.WhenLoadedWith<FileAvDataLoader>(
        AvPost.Do(static (_, _) => print("LOADED AVATAR FROM FILES"))
    ),
```

```
    AvPost.WhenLoadedWith<URIAvDataLoader>(
        AvPost.Do(static (_, _) => print("LOADED AVATAR FROM URIS"))
    ),

    AvPost.WhenImportedWith<UniVRM10AvImporter>(
        AvPost.Do(static (_, _) => print("LOADED VRM AVATAR"))
    ),
    AvPost.WhenImportedWith<UniGLTFAvImporter>(
        AvPost.Do(static (_, _) => print("LOADED UNIGLTF AVATAR"))
    ),
    AvPost.WhenImportedWith<GLTFastAvImporter>(
        AvPost.Do(static (_, _) => print("LOADED GLTFAST AVATAR"))
    )
).ProcessWithAsync(new CachePostProcessor(Application.temporaryCachePath))
.Build();
```

Handling Renders and Metadata

- Access `avatar.FullRender` and `avatar.BustRender` for thumbnails.
- `avatar.Metadata` includes ID, gender, type (e.g., `AvType.HumanoidFullBody`).

Cleanup

Always dispose loaded avatars:

```
avatar.Dispose();  // Destroys GameObject, renders, post-processor created
materials, etc.
```

For full API docs, please see https://uralstech.github.io/AvLoader/api/Uralstech.AvLoader.html or `APIReferenceManual.pdf` in the package documentation for the reference manual.

You can implement `IAvDataLoader` or `IAvImporter` for custom sources/importers (e.g., cloud APIs, FBX import).

Advanced - Capabilities

Capabilities are interfaces in the `Uralstech.AvLoader.Capabilities` namespace for `LoadedAv` that expose functionality guaranteed to exist because a the avatar importer created it, independent of later configuration or user modifications.

They act as a reliable contract so consuming code can safely detect and use importer-provided features, such as VRM LookAt, VRM Expressions, or an importer-generated Animator, without guessing or inspecting arbitrary components. Capabilities describe

availability and guarantees only, not ownership, configuration, or policy, and they are implemented exclusively by importer-backed `LoadedAv` objects, never by post-processors or user code.

Capabilities should *not* be accessed by using inferface methods directly, but by casting the `LoadedAv` object to the interface. You can also use the utility methods defined in `LoadedAv` for doing so:

```
LoadedAv avatar = ...
if (!avatar.TryGetCapability(out IBlendShapeProvider? provider))
    return;

if (provider.HasWeight("blink"))
    provider.SetWeight("blink", 1f); // Close the avatar's eyes.
```